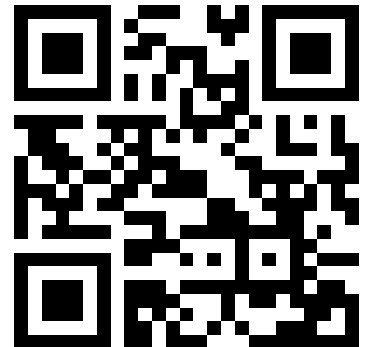


Advanced Micro-Controller Systems

- Introduction, Microcontroller basics
- • 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>



Selecting a Processor

- Computing Performance
- Memory options (RAM FLASH EEPROM)
- Scalability
- Environmental range (Temperature, radiation resistance etc.)
- I/O Options
- Power Consumption
- Development Environment
- Debugging Support
- Etc...

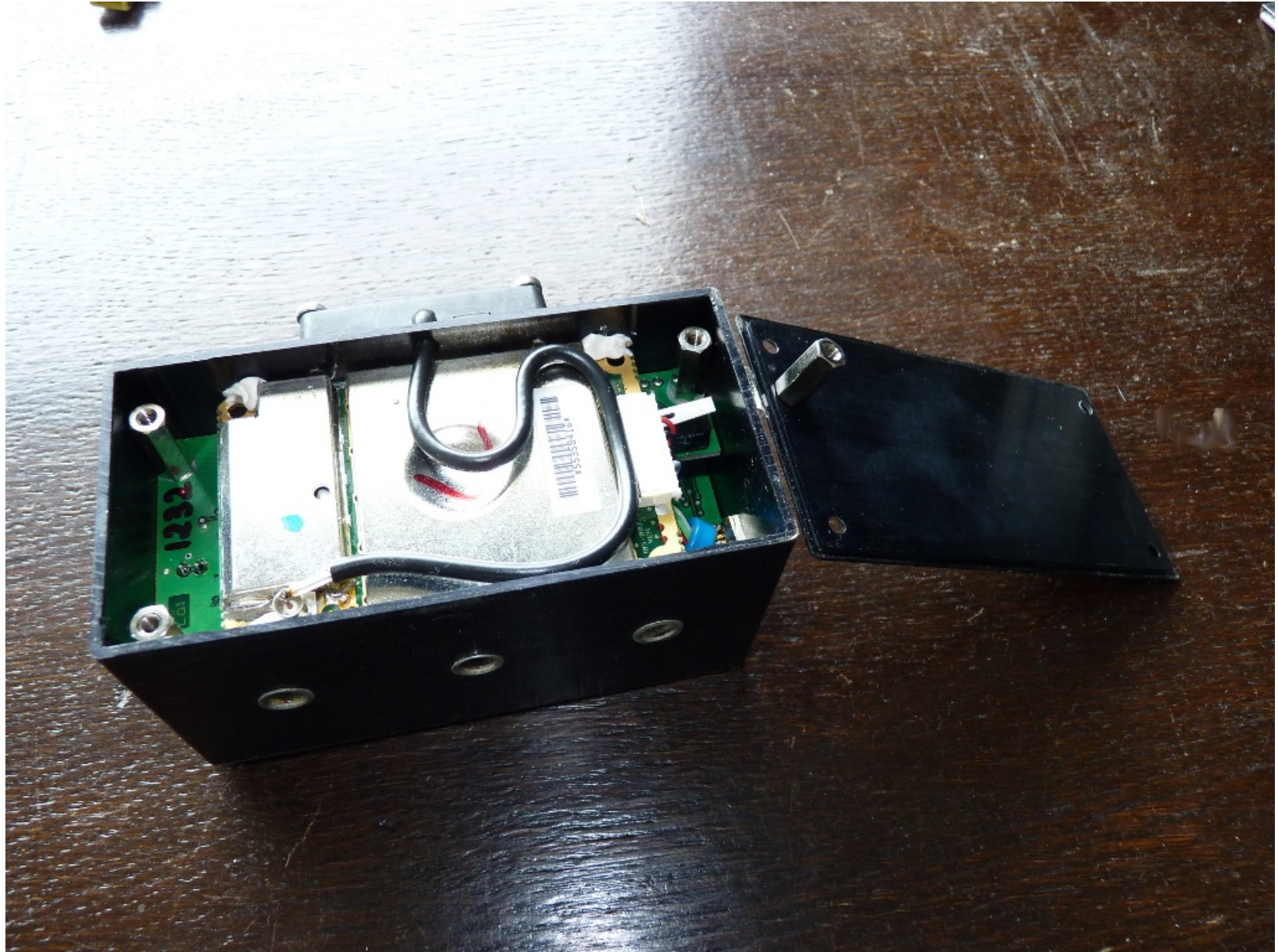
Performance Criteria

- Instructions per second (MIPS)
- Calculations per second (Benchmark)
- Calculations / second / MHz (rel. Benchmark)
- Calculations / Energy
Computation power / Electrical power
- (Interrupt-) Response time
- Quiescent Current (Battery-driven equipment !)
- Shutdown+Wakup - Energy
- Stability against harsh environment
(temperature, radiation etc.)

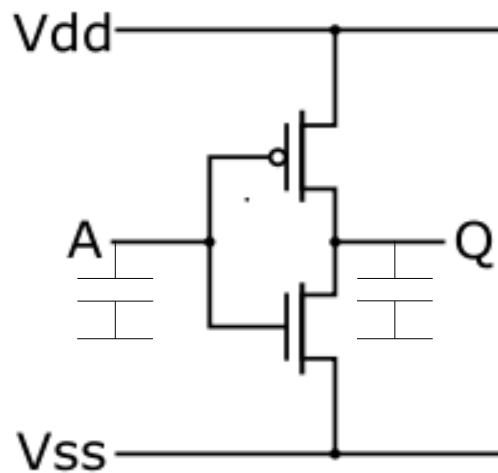
Special Requirements

- Non Maskable Interrupt (NMI)
- Watchdog (Detect hanging software)
- Brown-out detection (Detect hanging hardware)
- Sleep Modes (Save power in standby mode)
- Security:
 - Cryptography
 - Random Number Generator
- Security of code (Read-out protection)
- Graphics / Video capability
- Integrated radio hardware
- Anti-Tampering features

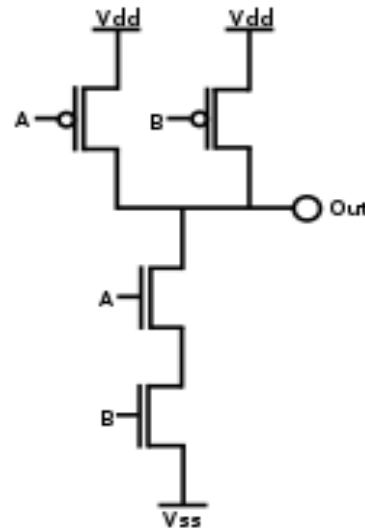
BTW: Anti-Tamper Switch



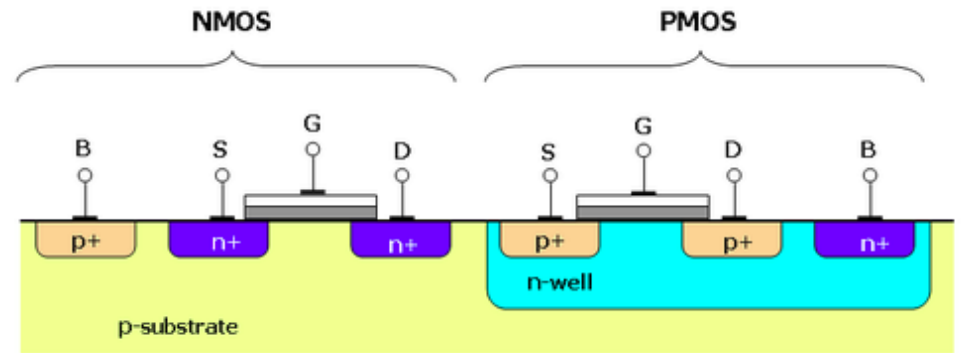
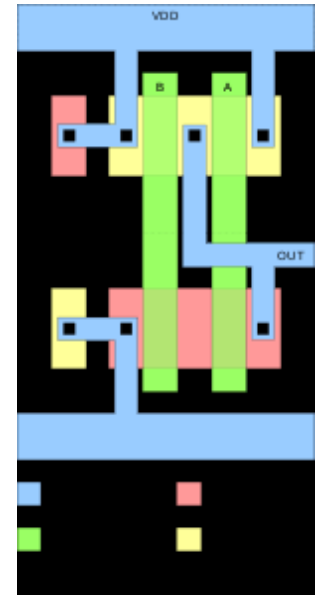
"What is" ... mA / MHz ???



NOT



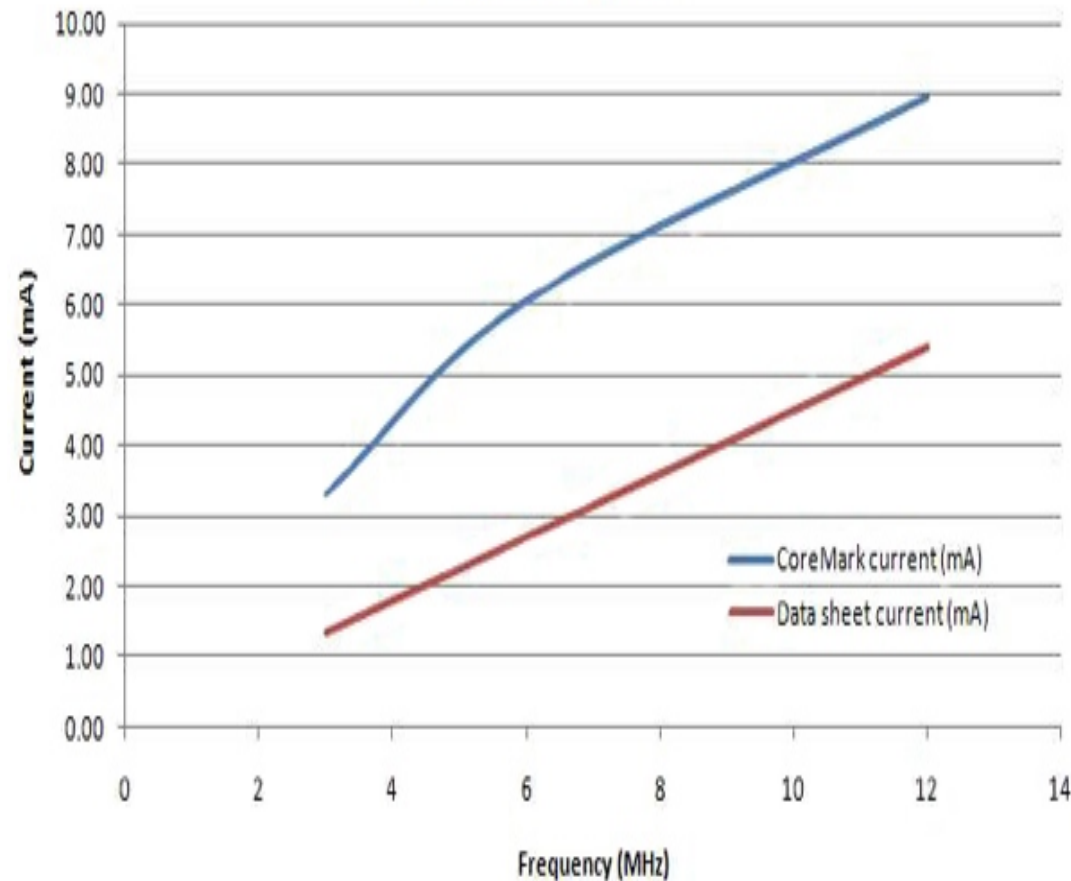
NAND



Current consumption

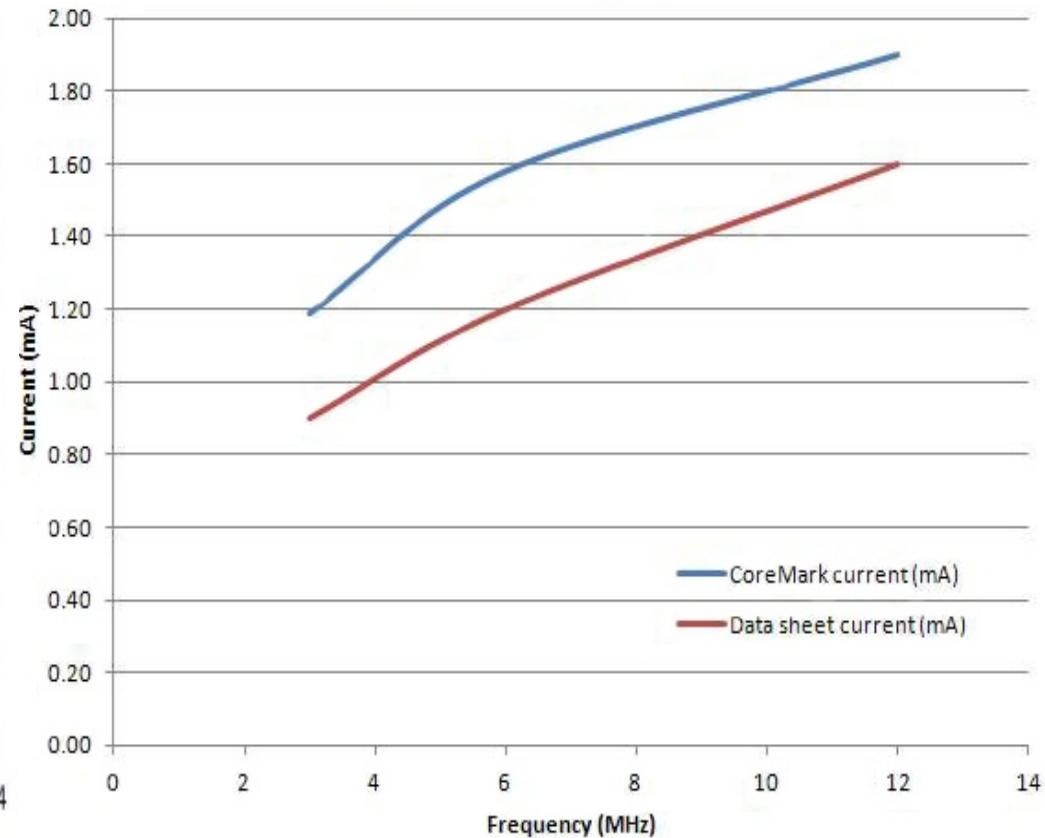
8bit vs 32bit Microcontroller

Current versus Frequency

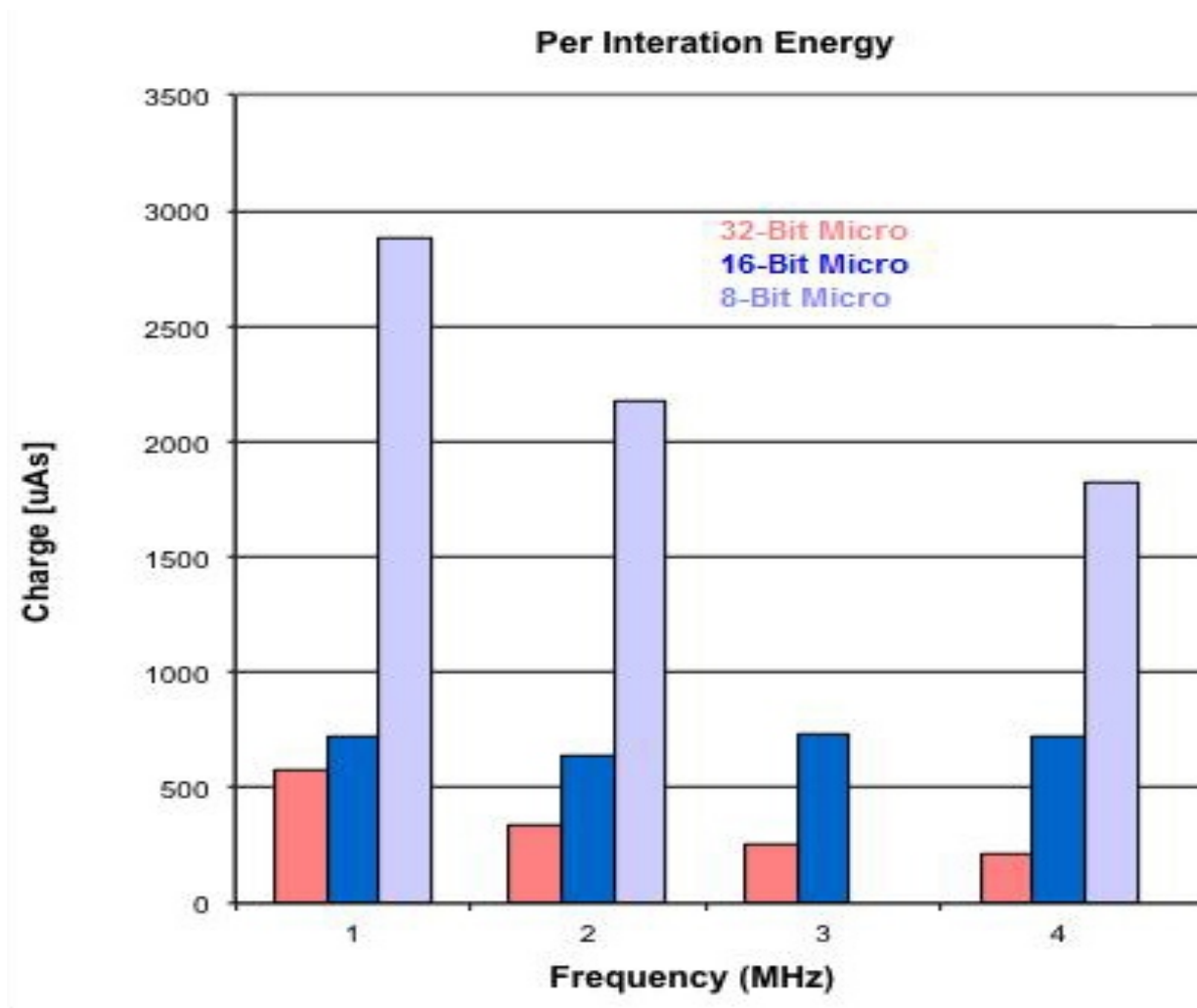


8 bit

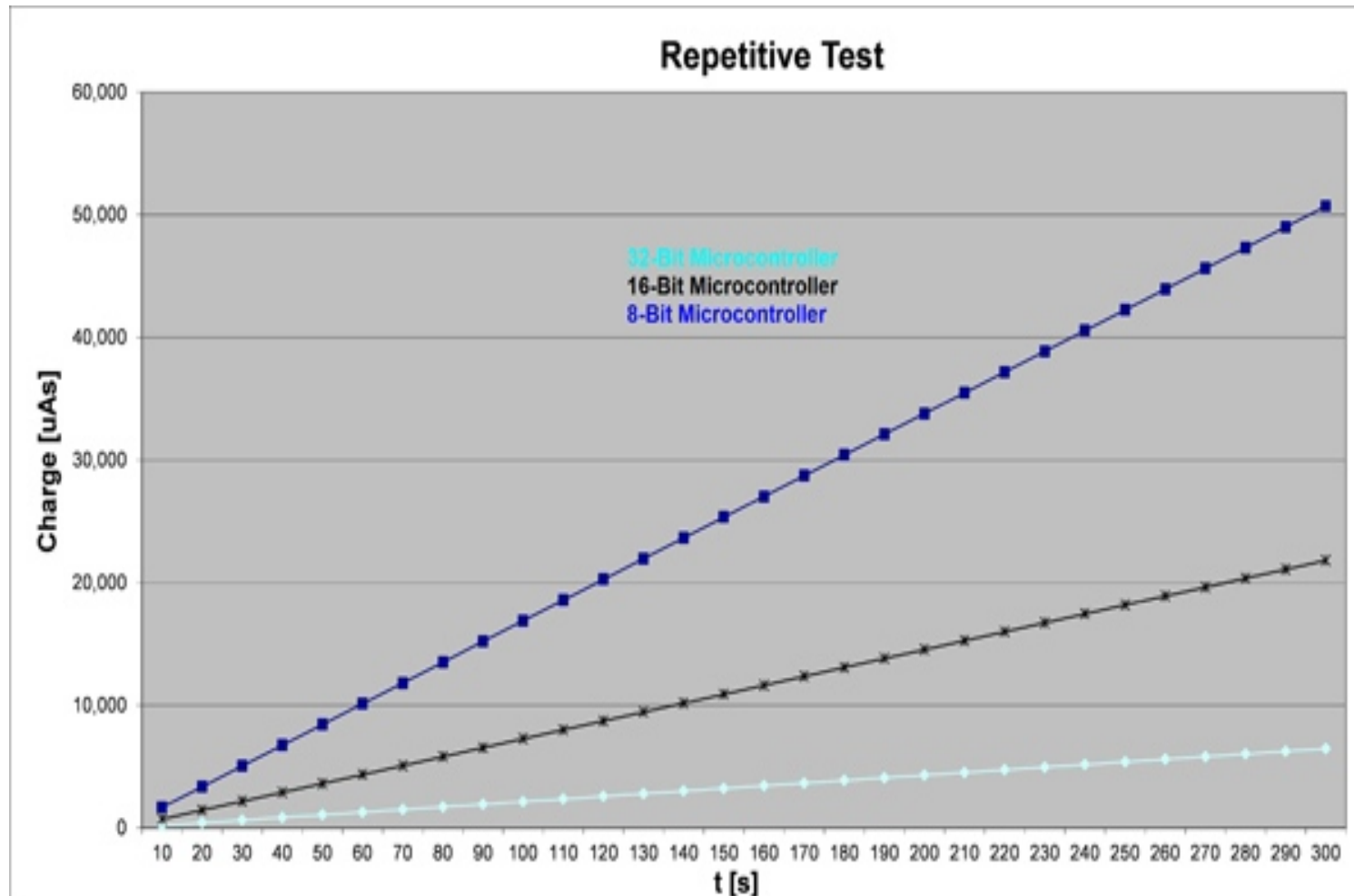
Current versus Frequency



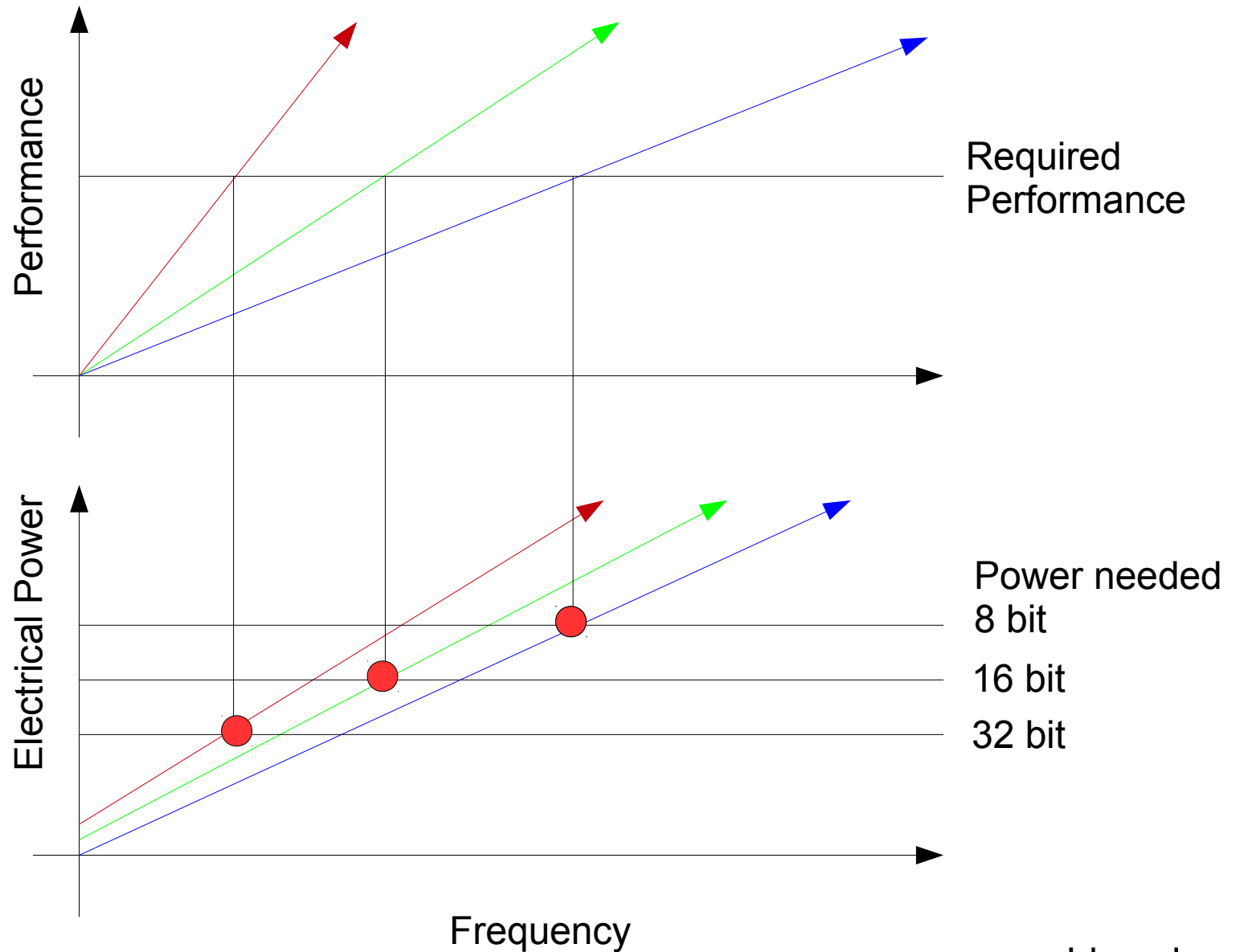
32bit



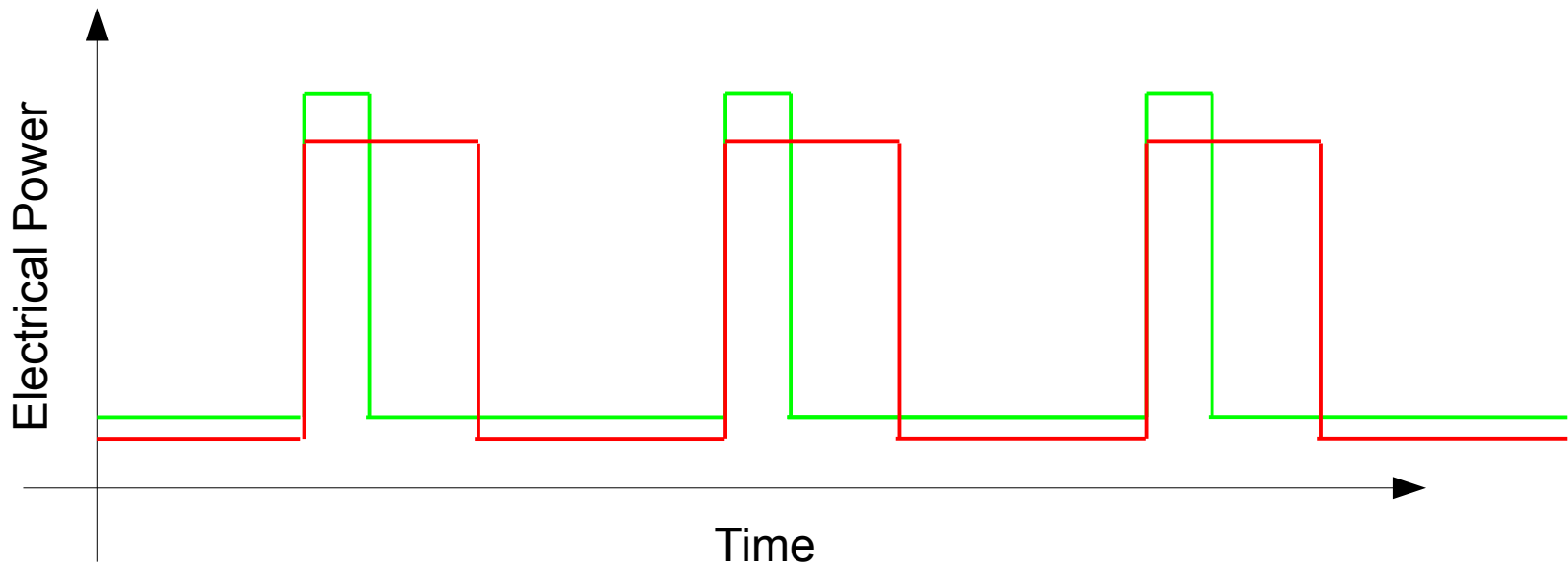
Battery Lifetime



Computation Power vs Electrical Power



Electrical Power Over Time



Crucial criteria are

- Quiescent Power Consumption
- Active Power consumption
- Benchmark-Performance of the Processor

Performance

What is it ?

Computation output / time

Computation output / energy

Response time

Reliability

...

Benchmarks

Test performance under realistic conditions

Compare different architectures and models

Choose the best design for a specific application

Tests for special applications include:

- Integer computation
- Floating-point computation, DSP
- Addressing arrays
- Bit manipulation
- Input / Output operations
- Response time (Interrupt latency)
- Cryptography ...

Linpack

LINPACK is a collection of Fortran subroutines that analyze and solve linear equations and linear least-squares problems.

The package solves linear systems whose matrices are general, banded, symmetric indefinite, symmetric positive definite, triangular, and tridiagonal square.

In addition, the package computes the QR and singular value decompositions of rectangular matrices and applies them to least-squares problems.

LINPACK uses column-oriented algorithms to increase efficiency by preserving locality of reference.

Whetstone

Mixture of algorithms for different purposes

```
X = 0.5;
Y = 0.5;

for (I = 1; I <= N7; I++)
{
    X = T * DATAN(T2*DSIN(X)*DCOS(X)/(DCOS(X+Y)+DCOS(X-Y)-1.0));
    Y = T * DATAN(T2*DSIN(Y)*DCOS(Y)/(DCOS(X+Y)+DCOS(X-Y)-1.0));
}
```

```
J = 1;
K = 2;
L = 3;

for (I = 1; I <= N6; I++)
{
    J = J * (K-J) * (L-K);
    K = L * K - (L-J) * K;
    L = (L-K) * (K+J);
    E1[L-1] = J + K + L;
    E1[K-1] = J * K * L;
}
```

```
J = 1;
for (I = 1; I <= N4; I++) {
    if (J == 1)
        J = 2;
    else
        J = 3;

    if (J > 2)
        J = 0;
    else
        J = 1;

    if (J < 1)
        J = 1;
    else
        J = 0;
}
```

```
E1[1] = 1.0;
E1[2] = -1.0;
E1[3] = -1.0;
E1[4] = -1.0;

for (I = 1; I <= N2; I++)
{
    E1[1] = ( E1[1] + E1[2] + E1[3] - E1[4]) * T;
    E1[2] = ( E1[1] + E1[2] - E1[3] + E1[4]) * T;
    E1[3] = ( E1[1] - E1[2] + E1[3] + E1[4]) * T;
    E1[4] = (-E1[1] + E1[2] + E1[3] + E1[4]) * T;
}
```

Dhrystone “DMIPS”

Integer operations

String manipulation, searching, sorting

```
void Proc8(Array1Par, Array2Par, IntPar1, IntPar2)
Array1Dim Array1Par;
Array2Dim Array2Par;
OneToFifty IntPar1;
OneToFifty IntPar2;
{
    REG OneToFifty IntLoc;
    REG OneToFifty IntIndex;
    IntLoc = IntPar1 + 5;
    Array1Par[IntLoc] = IntPar2;
    Array1Par[IntLoc + 1] = Array1Par[IntLoc];
    Array1Par[IntLoc + 30] = IntLoc;
    for (IntIndex = IntLoc; IntIndex <= (IntLoc + 1); ++IntIndex)
    {
        Array2Par[IntLoc][IntIndex] = IntLoc;
        ++Array2Par[IntLoc][IntLoc - 1];
        Array2Par[IntLoc + 20][IntLoc] = Array1Par[IntLoc];
        IntGlob = 5;
    }
}
```

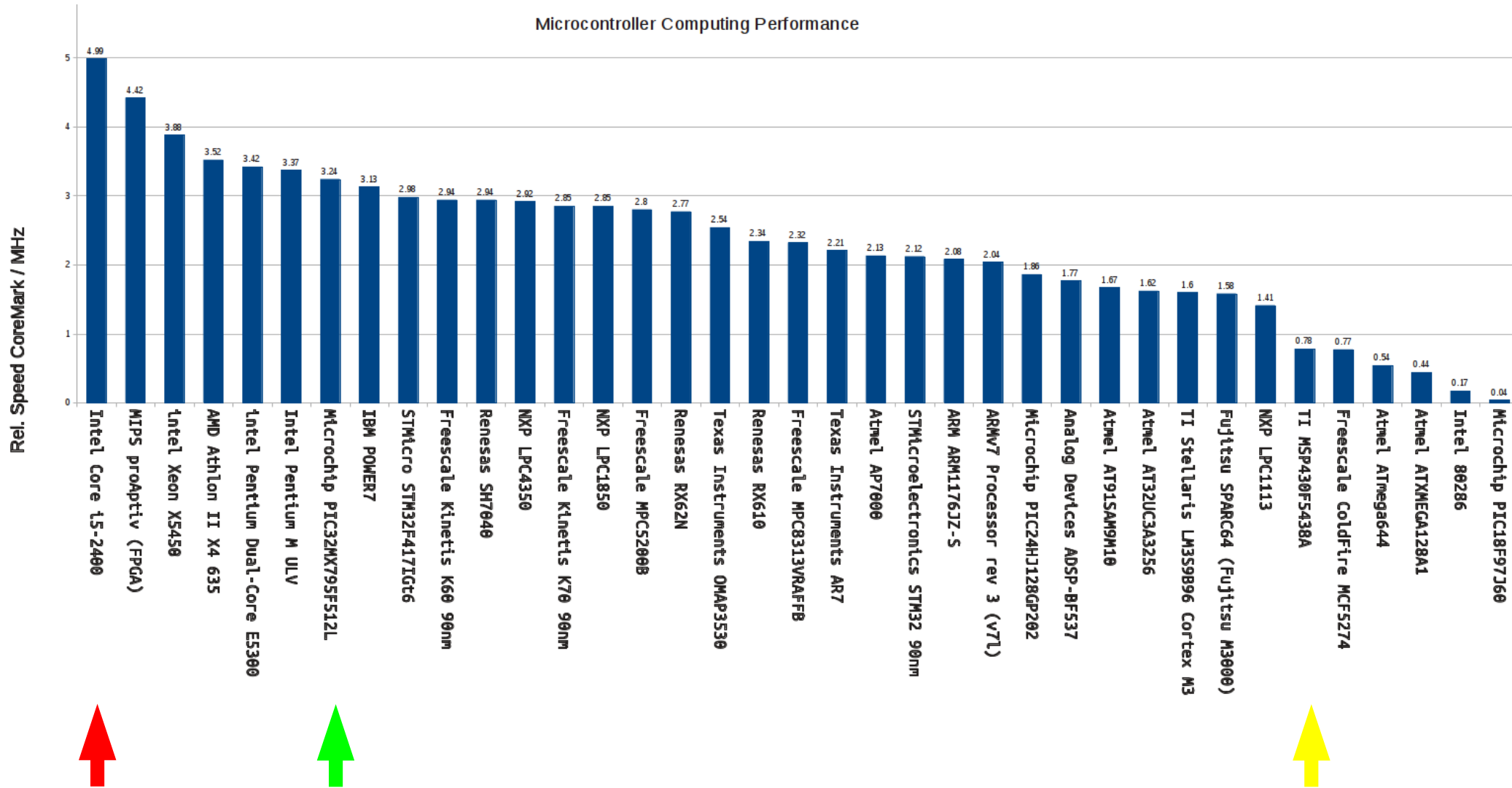
Coremark

- Special benchmark targeted for embedded microcontrollers
- Linked lists, pointers, array data
- Vectors and matrices, Integer computation
- State-machine implementation
- Cyclic redundancy check computation

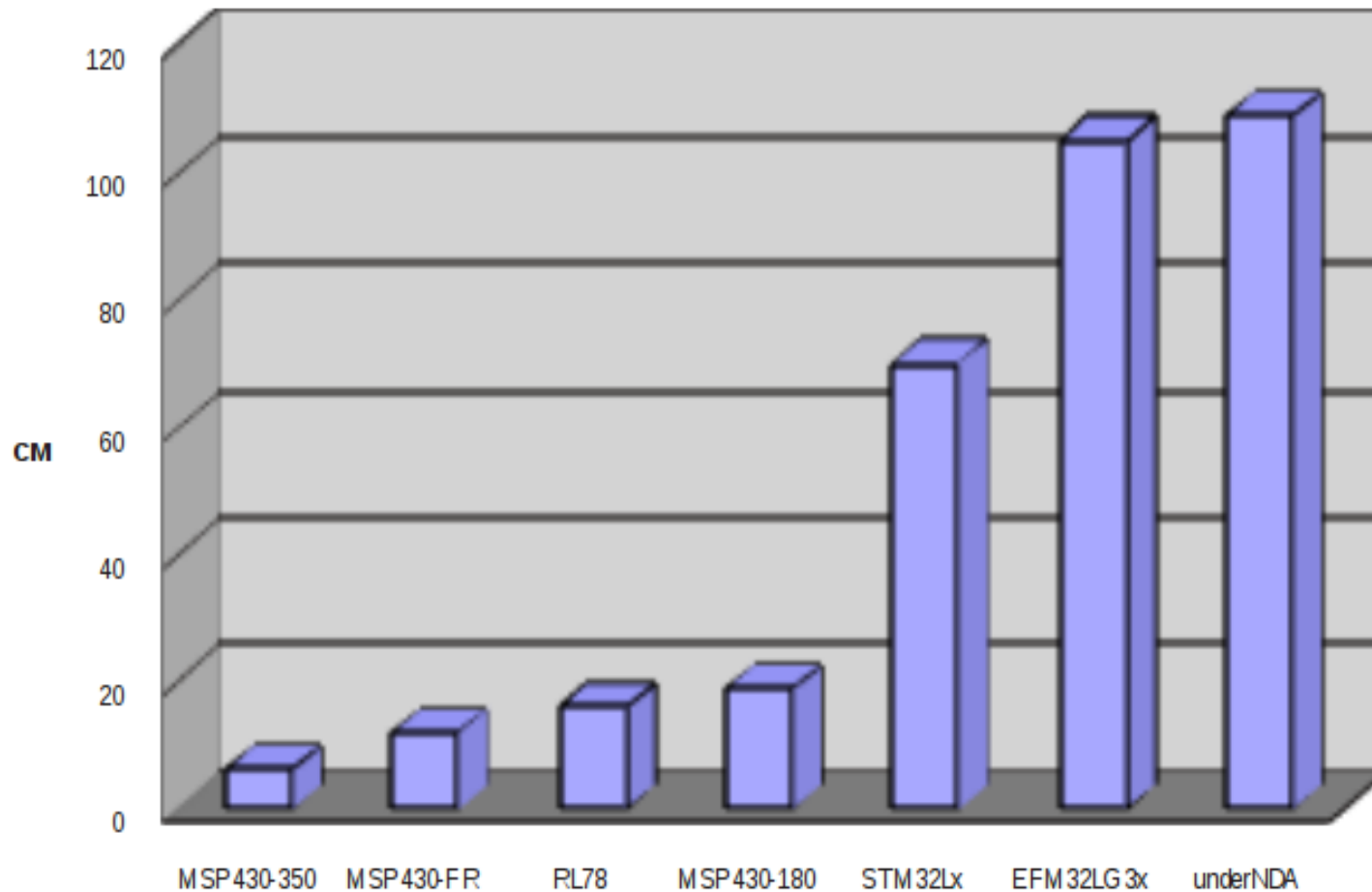
CoreMark[®]

An EEMBC Benchmark

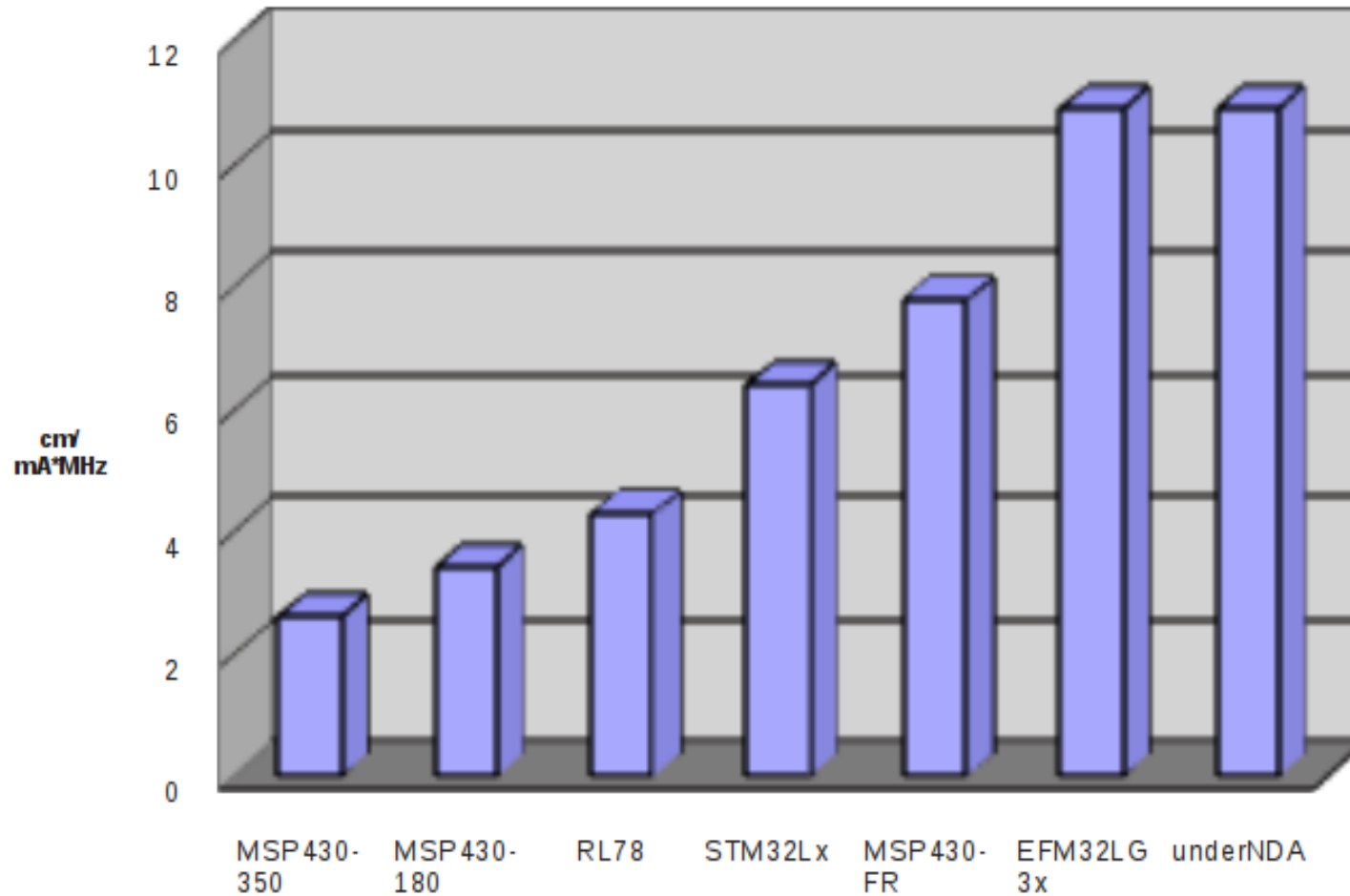
Coremark / MHz



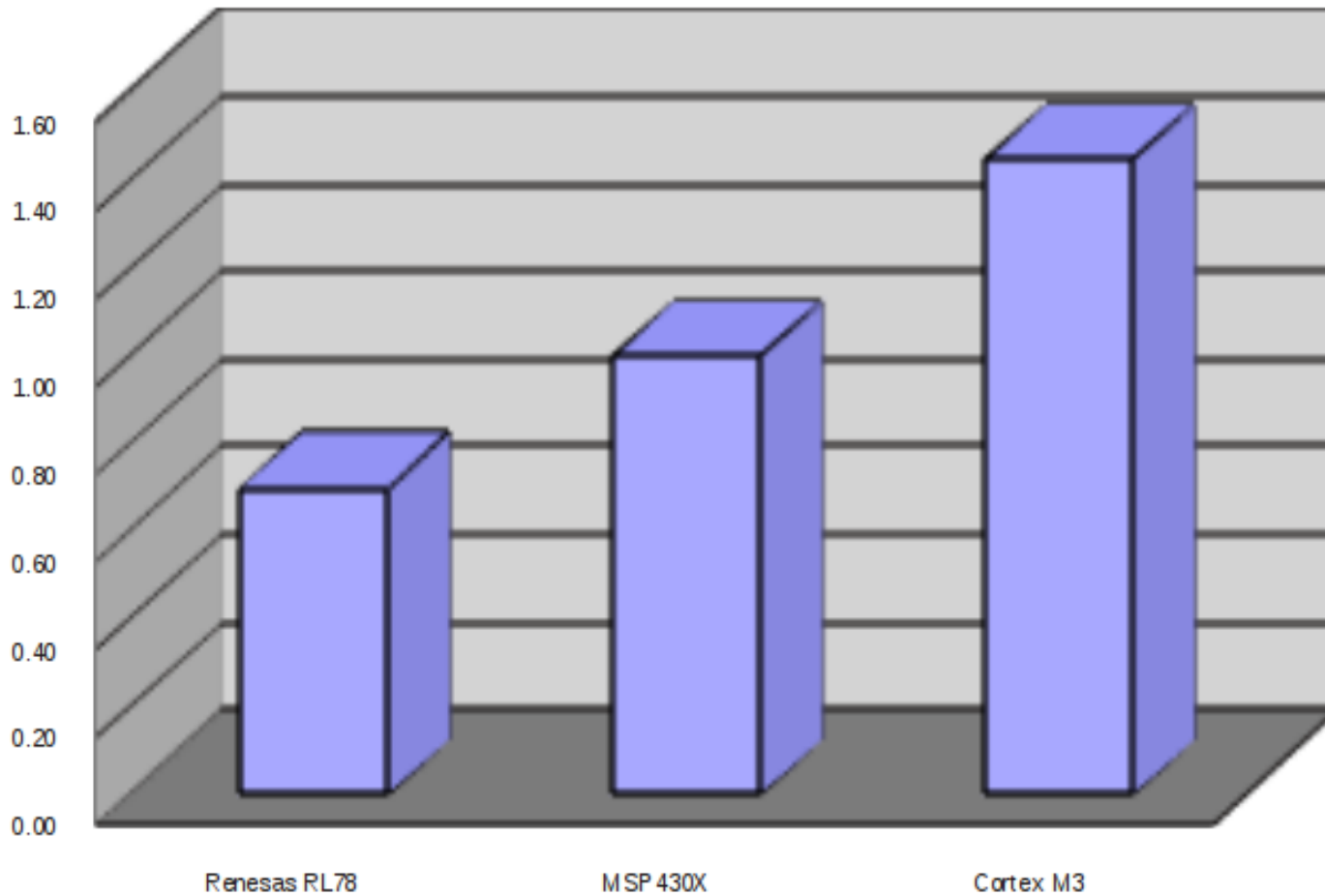
Coremark Performance



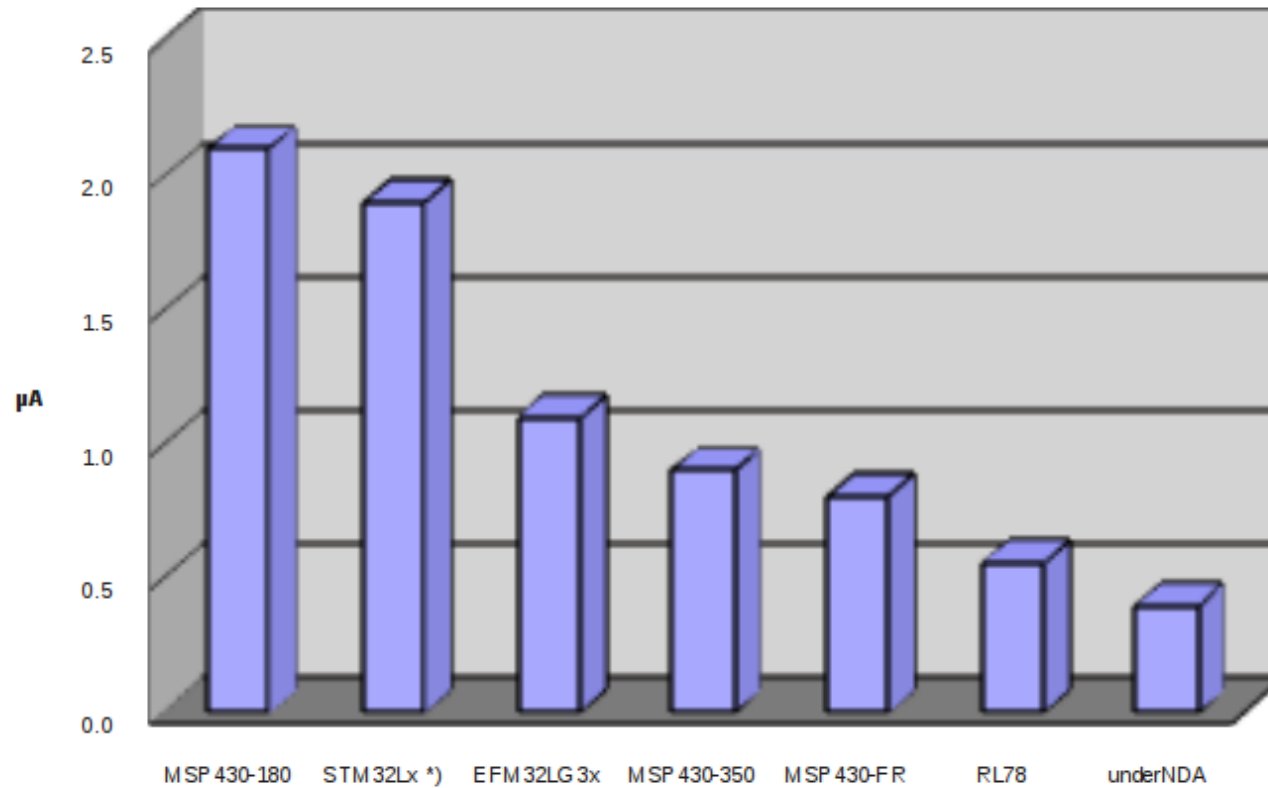
Coremark / (mA * MHz)



Code efficiency (float computation)



Quiescent Power



Source: Dipl. Ing Winfried Simon, priv. comm.